CS 360
Programming Languages
Day 24

Review

- Parsers and Grammars
- Lexers and Tokenizers

Lexemes and Tokens

- Lexemes are the smallest “lexical units” in a language:
  - 101
  - my-var
  - +

- Tokens are effectively the same thing, though there is a difference (perhaps more on that later)

Lexers and Tokenizers

- The tokenizer is a function which takes in raw source code and breaks it into parse-able “tokens” (or lexemes)

- In our context (today), the lexer will be a helper function for the tokenizer

Where We Are

- We have a working parser/grammar for bf. This produces a *parse tree* for any program.

- **Today:** We need a tokenizer, which handles white space and comments, catches/handles invalid commands/tokens.

- **Next Class:** We need an expander: what does a parse tree mean? (Semantics!)

lexer

- We'll use a function called `lexer` which takes in pairs:

  `[trigger action-expr]`
- `trigger` is a pattern / regular expression or end-of-file character
- `action-expr` is something we should interpret the current `trigger` as: for now, this will just be `lexeme`

- Our lexer needs to match our defined commands and ignore anything else!
- **If we want to extend our language with new commands/syntax, we'll have to modify the lexer!**

Questions about the Lexer / Tokenizer

- What does the bf-lexer function actually do?
- What is next-token? How/when does it get called?
- How does the parse function work?
- Come up with at least one question about the lexer/tokenizer!
  (I may not know the answer!!)

Next: The Expander

- Let’s brainstorm how the expander should work!
- Remember: this is where we encode bf behavior in Racket code.
- What do we need?